计 算 机 体 系 结 构

第十一讲

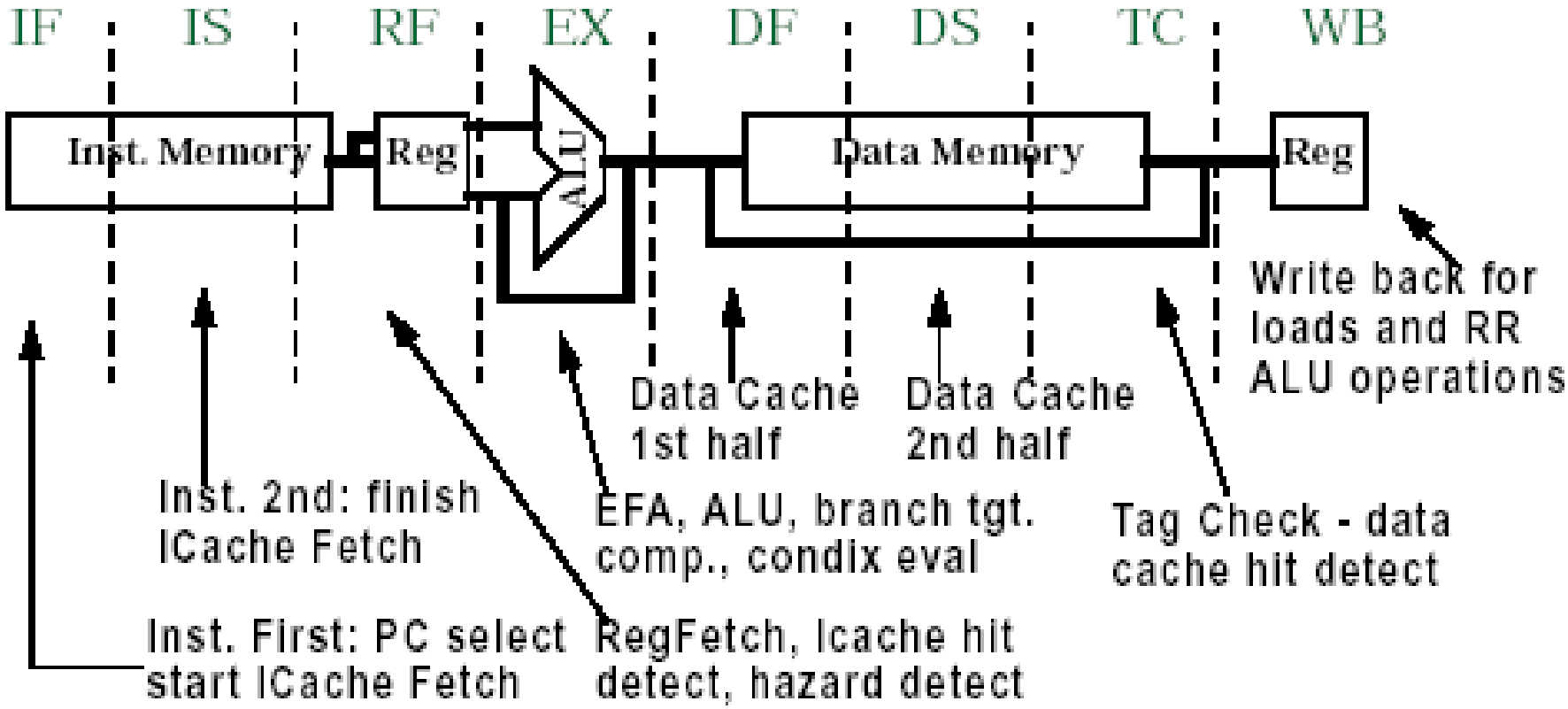
计算机科学与技术学院
舒燕君

Recap

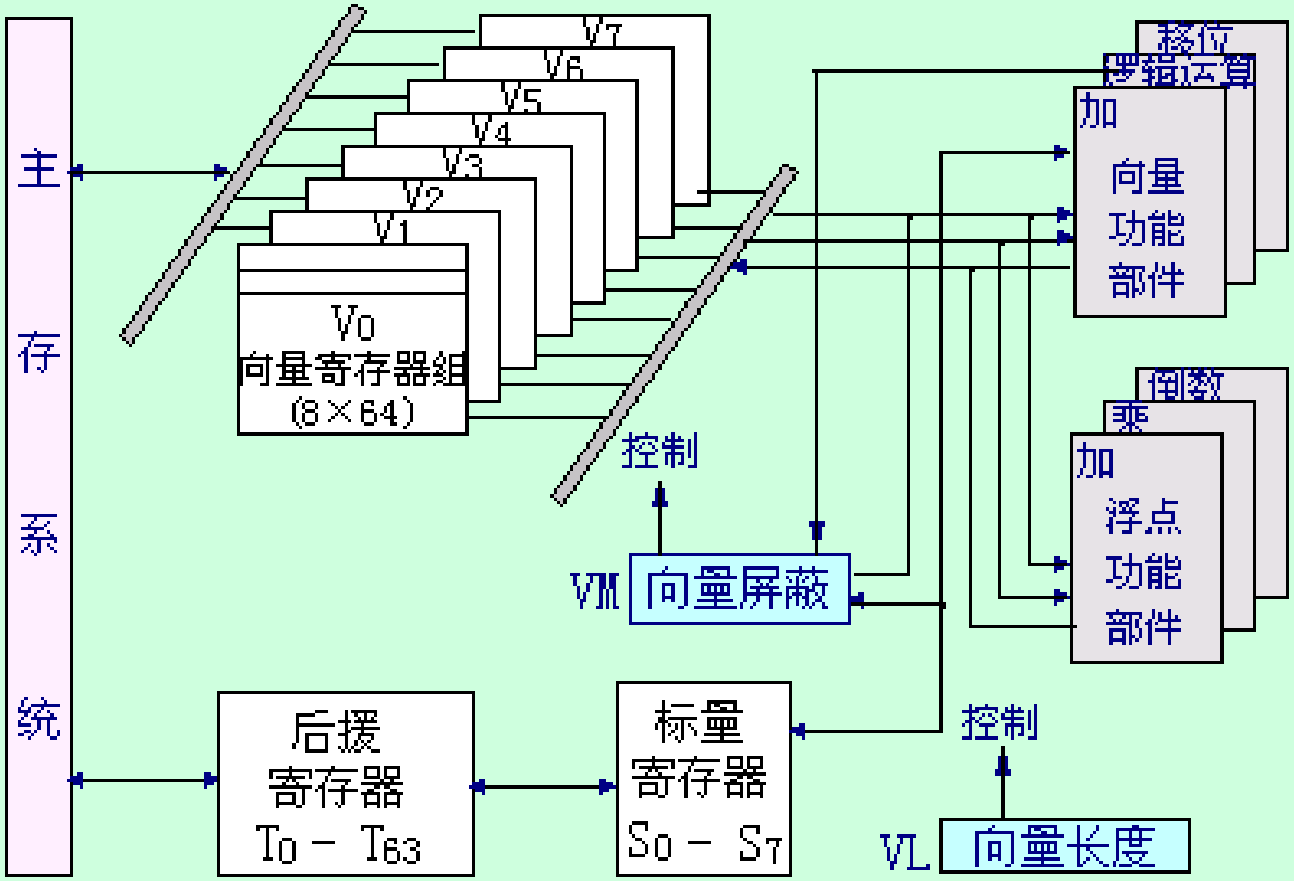
- 流水线的控制冲突
  - ✓ 减少流水线分支损失的方法（冻结或排空、预测分支转移失败、预测分支转移成功、分支延迟槽）
- 流水线的实例（**MIPS R4000**）
  - ✓ 整型流水线（深流水、**8**段）
  - ✓ 浮点流水线（多功能非线性流水线）
  - ✓ 流水线性能分析
- 向量处理机 **Cray-I**
  - ✓ 性能指标、基本结构
  - ✓ 链接技术

三种方法的要求及效果

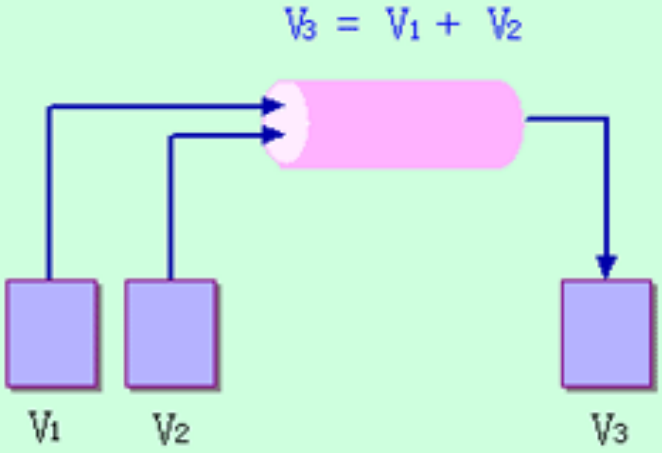
调度策略	对调度的要求	其作用前提
从前调度	被调度的指令必须与分支结果无关	任何情况
从目标处调度	必须保证在分支失败时执行被调度的指令不会导致错误，可能需要复制指令	分支成功时
从失败处调度	必须保证在分支成功时执行被调度的指令不会导致错误	分支失败时

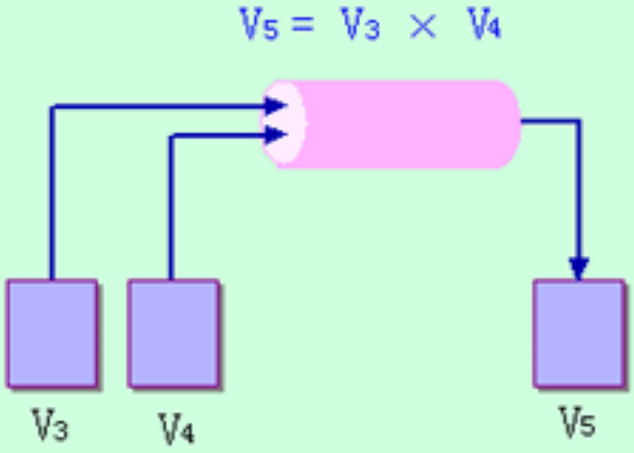

IF
IS
RF
EX
DF
DS
TC
WB
Inst. Memory
Reg
ALU
Data Memory
Reg
Write back for loads and RR ALU operations
Data Cache 1st half
Data Cache 2nd half
Inst. 2nd: finish ICache Fetch
EFA, ALU, branch tgt. comp., condix eval
Tag Check - data cache hit detect
Inst. First: PC select start ICache Fetch
RegFetch, Icache hit detect, hazard detect

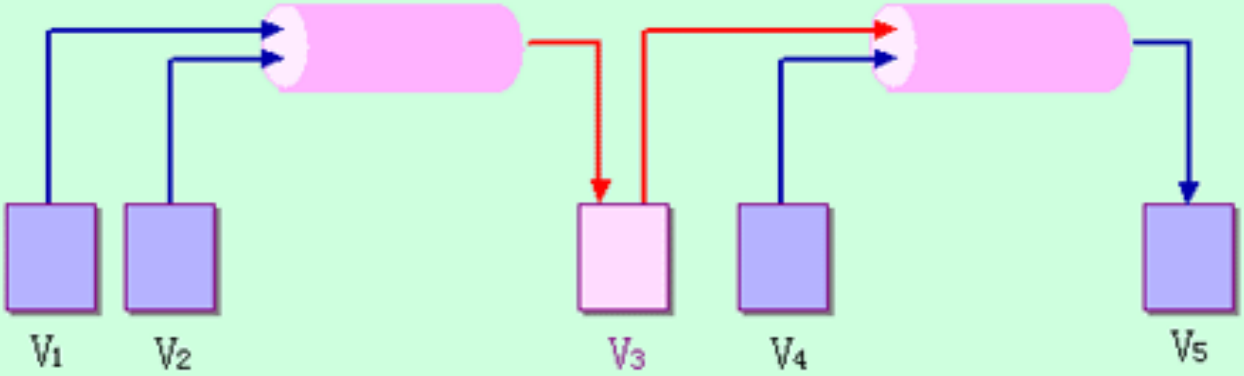
CRAY-1的基本结构


主
存
系
统
V7
V6
V5
V4
V3
V2
V1
V0
向量寄存器组
(8×64)
移位
逻辑运算
加
向量
功能
部件
倒数
乘
加
浮点
功能
部件
控制
VM
向量屏蔽
后援
寄存器
T0 – T63
标量
寄存器
S0 – S7
控制
VL
向量长度

链接特征


$V_3 = V_1 + V_2$
V_1
V_2
V_3


$V_5 = V_3 \times V_4$
V_3
V_4
V_5


$V_3 = V_1 + V_2$
$V_5 = V_3 \times V_4$
V_1
V_2
V_3
V_4
V_5

向量机总结

- 向量机适合挖据规整的数据级并行
  - 同样的操作作用在许多数据元素上
  - 提高性能、设计简单（向量内的操作相互独立）
- 性能的提升受限于代码的向量化
  - 标量操作限制着向量机的性能
- 很多已有的ISA扩展了一些SIMD操作
  - **Intel MMX/SSEn/AVX, PowerPC AltiVec, ARM Advanced SIMD**
- 另一种SIMD变体，图形处理单元GPU

Graphics Processing Units (GPUs)

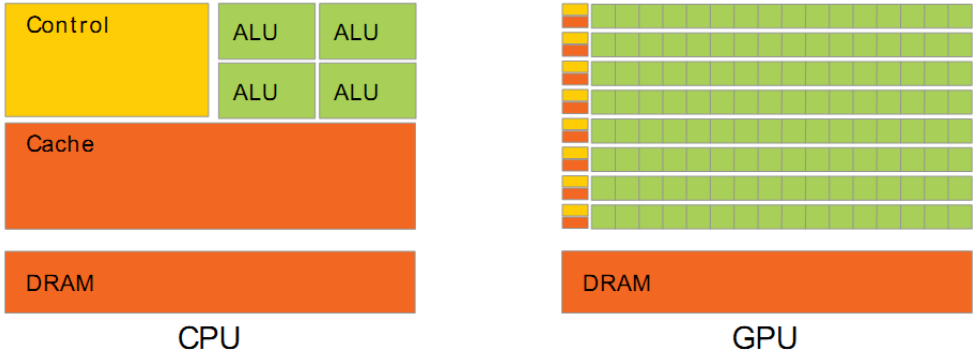
- 早期的GPU是指带有高性能浮点运算部件、可高效生成3D图形的具有固定功能的专用设备 (mid-late 1990s)
  - 让PC机具有类似工作站的图形功能
  - 用户可以配置图形处理流水线，但不是真正的对其编程
- 2001-2005，GPU加入了越来越多的可编程性
  - 例如新的语言 Cg可用来编写一些小的程序处理图形的顶点或像素，是Windows DirectX的变体
  - 大规模并行（针对每帧上百万顶点或像素）但非常受限于编程模型
- 通过将输入和输出数据映射为图像，并对顶点或像素渲染计算 可进行通用计算
  - 因为不得不使用图形流水线模型，这对完成通用计算来说是个非常难用的编程模型

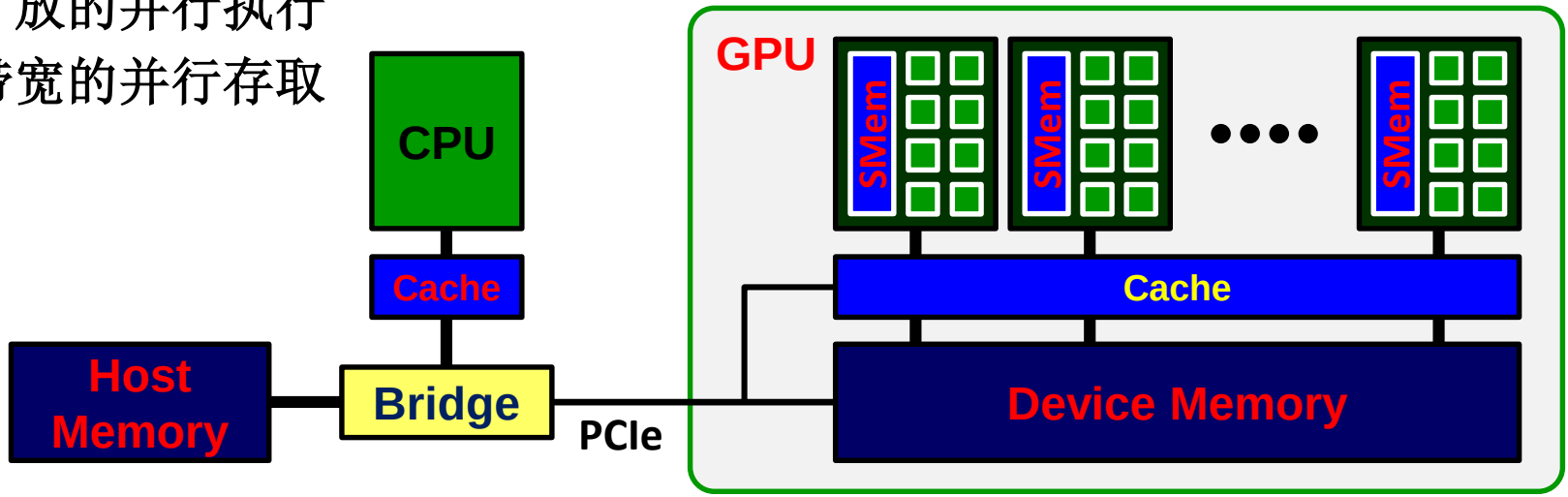
General-Purpose GPUs (GP-GPUs)

- **2006年, Nvidia 的 GeForce 8800 GPU 支持一种新的编程语言: CUDA**
  - **“Compute Unified Device Architecture”**
  - **随后工业界推出OpenCL，与CUDA具有相同的ideas, 但独立于供应商**
- **Idea: 针对通用计算，发挥GPU计算的高性能和存储器的高带宽来加速一些通用计算中的核心（Kernels）**
- **一种附加处理器模型（GPU作为附加设备）：Host CPU发射数据并行的kernels 到GP-GPU上运行**
- **仅讨论Nvidia CUDA样式的简化版本，仅考虑GPU的计算核部分，不涉及图形加速部分**

Using CPU+GPU Architecture

- **CPU+GPU异构多核系统**
  - 针对每个任务选择合适的处理器和存储器
- **通用CPU 适合执行一些串行的线程**
  - 串行执行快
  - 带有cache，访问存储器延时低
- **GPU 适合执行大量并行线程**
  - 可扩放的并行执行
  - 高带宽的并行存取





CPU与GPU的性能比较

	Num. PE	Frequency (GHz)	Num. Transistors	BW (GB/sec)	SP SIMD width	DP SIMD width	Peak SP Scalar FLOPS (GFLOPS)	Peak SP SIMD Flops (GFLOPS)	Peak DP SIMD Flops (GFLOPS)
Core i7-960	4	3.2	0.7B	32	4	2	25.6	102.4	51.2
GTX280	30	1.3	1.4B	141	8	1	116.6	311.1/933.1	77.8

Table 2: Core i7 and GTX280 specifications. BW: local DRAM bandwidth, SP: Single-Precision Floating Point, DP: Double-Precision Floating Point.

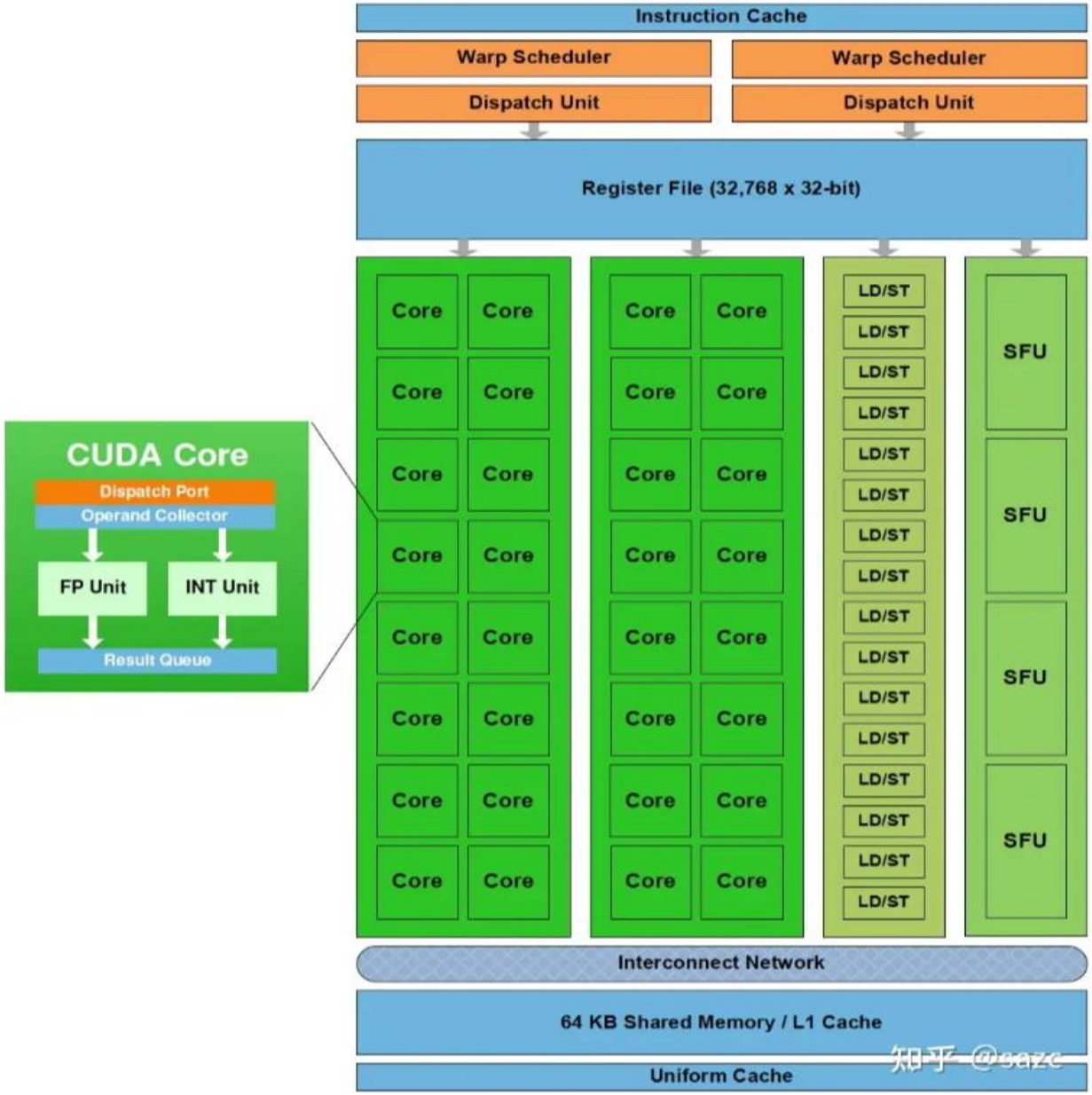
Apps.	SGEMM	MC	Conv	FFT	SAXPY	LBM	Solv	SpMV	GJK	Sort	RC	Search	Hist	Bilat
Core i7-960	94	0.8	1250	71.4	16.8	85	103	4.9	67	250	5	50	1517	83
GTX280	364	1.4	3500	213	88.8	426	52	9.1	1020	198	8.1	90	2583	475

Table 3: Raw performance measured on the two platforms. From the left, metrics are Gflops/s, billion paths/s, million pixels/s, Gflops/s, GB/s, million lookups/s, FPS, Gflops/s, FPS, million elements/s, FPS, million queries/s, million pixels/s, million pixels/s.

CSDN @GutsShinyHero

GPU中的SM

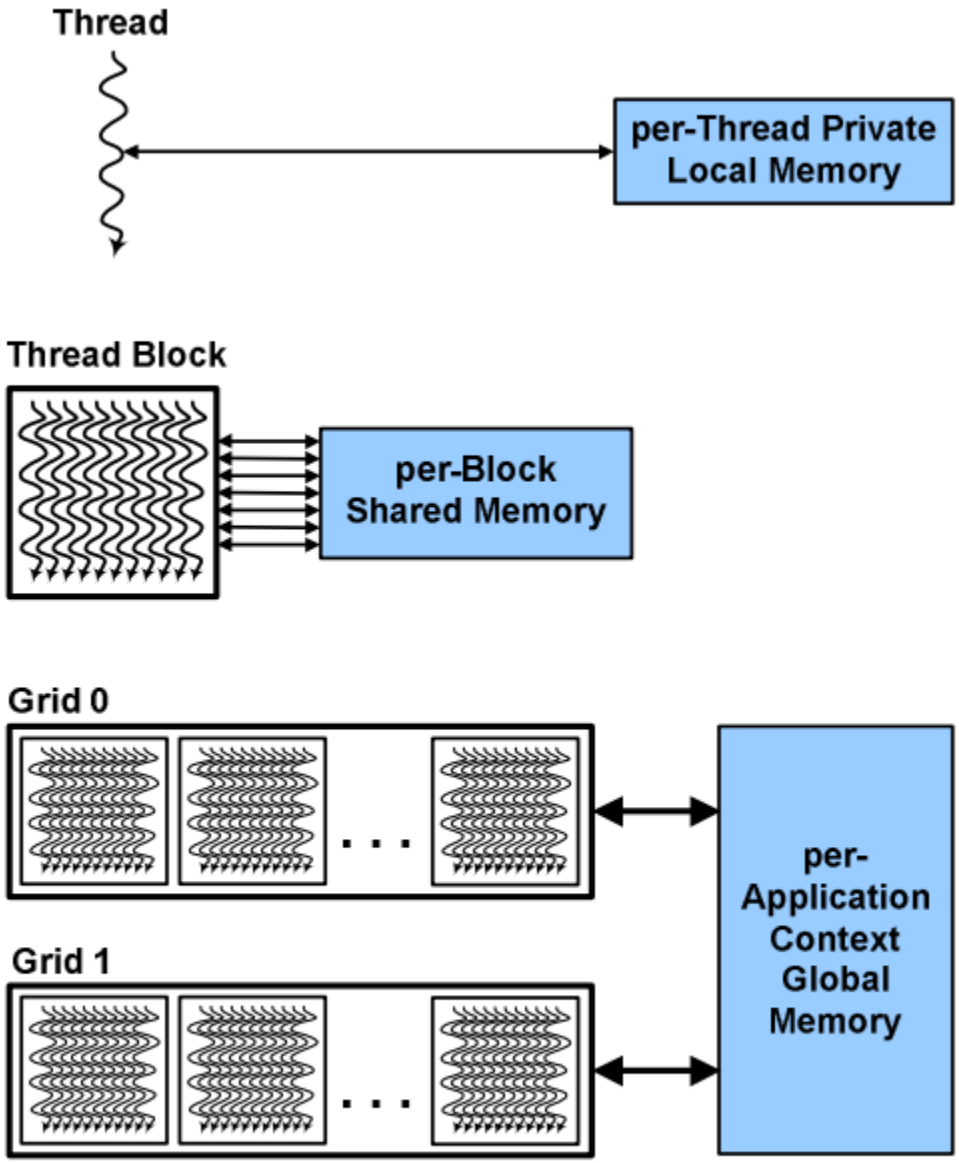
- **SM（Streaming Multiprocessor）**
  - 每个包含多个CORE及特殊函数单元（Special Function Unit，SFU）、寄存器、共享内存和常量缓存。
  - 在执行高密度计算时，每一个SM都可以实现数以千计个线程的并发执行，当核函数被启动时，GPU会根据分配策略将线程块分配到相应的SM执行，但是不同架构下单个SM的常驻线程数也不尽相同。



Fermi Streaming Multiprocessor (SM)

GPU中的SIMT

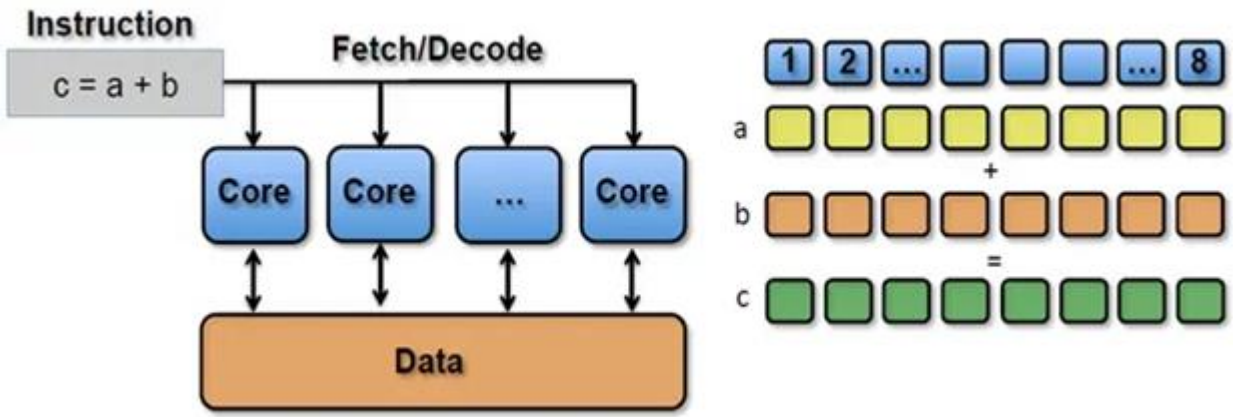
- 单指令多线程SIMT
  - GPU中指令并行技术SIMT ，是SIMD的升级版
  - 单条指令可以被多个线程同时执行。指令在不同线程中同时被执行时，其中存取、操作的数据可以是不同的。
  - 在使用时，会以SM为单位，SM单元是指令执行的“最小完整单元”。
  - 单个SM中的多个Core同时处理同一指令，每个core上运行一个线程，并且每个Core存取的数据可以是不同的。



CUDA Hierarchy of threads, blocks, and grids, with corresponding per-thread private, per-block shared, and per-application global memory spaces.

GPU中的SIMT

- **SIMT_ADD c, a, b**
  - 指令会被同时送入在单个SM中被编组的所有Core中，同时执行运算，但a、b 、c的值可以不一样



GPU中的线程束

- 线程束
  - 线程束的大小，即一个线程束中线程的数量，由硬件的计算能力版本决定。在目前所有的NVIDIA GPU中，一个线程束由连续的32个线程组成。
  - 一个SM中有32个core，当一个线程束执行时，线程束中的32个线程会被分配到32个core中，每个core中单独运行一个线程，32个core运行相同的指令。
  - 线程束的好处就是，一个warp只需要一个套逻辑对指令进行解码和执行就可以了，芯片可以做的更小更快，之所以可以这么做是由于GPU需要处理的任务是天然并行的。

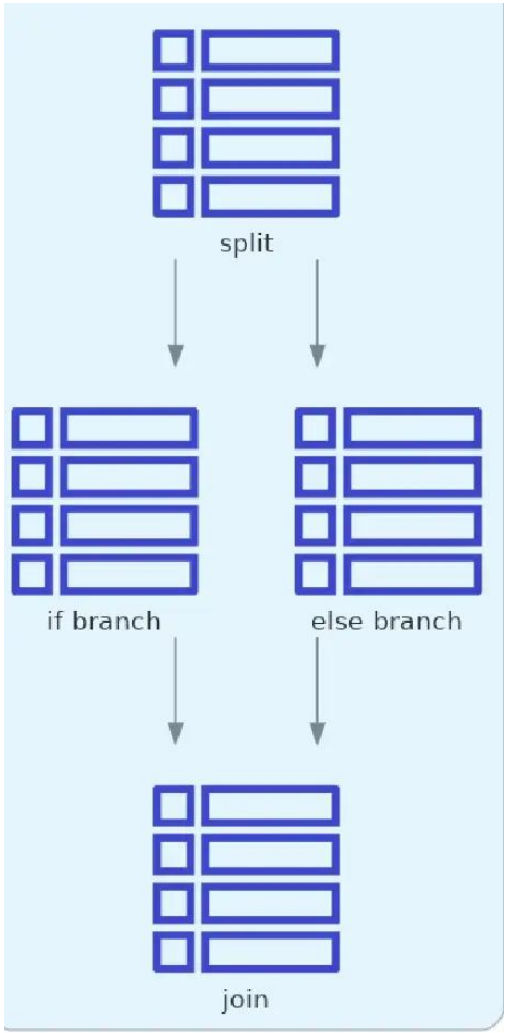
GPU中的控制相关

- 线程束中的分支
  - 线程束内的线程共享同样的PC，以锁步的方式执行指令，但是每个线程又可以有自己的执行分支。虽然允许每个线程有不同的分支，但是当一个线程束中出现不同分支的线程时，会大大降低效率。
  - 如果一个warp中需要进行分支（即warp内的线程执行的指令指针指向不同的位置），性能将急剧的下降。如果一个warp内需要执行N个分支，那么SM就需要把每一个分支的指令发射到每一个core上，再由core根据线程的逻辑决定需不需要执行。这是一个串行过程，因此SIMT完成分支的时间是多个分支时间之和。

GPU中的控制相关

- 线程束中的分支
  - 假设一个**Warp**中有**16**个线程判断条件为真，另外**16**个线程条件为假，所以一半线程会执行**if**中的语句，另一半线程执行**else**中的语句。
  - 假设每个分支任务量大致相同，分支分歧造成的性能损失少则原先的一半，最坏的情况如果每个线程执行分支都不一致，性能下降为最高时候的**1/32**。

- 线程束中的分支
  - **NVIDIA使用了SIMT Stack的**方式来处理线程束中的分支问题
  - 在硬件层面增加两条指令：**split join**，用于在并行计算中实现线程束的分割和合并操作。
  - 这些指令能够将线程束分为更小的子线程束，并在之后将它们合并成更大的线程束。在执行分支时，另一个分支的信息需要保存，因此引入了**SIMT Stack**。



branch example

本章小结

1. 流水线的基本概念和分类
   - 流水技术的特点和流水线定义
   - 五种分类
2. 流水线性能的主要指标
   - 吞吐率、加速比、效率
   - 实际值和最大值
3. 流水线冲突
   - 结构冲突（瓶颈段细分、重复设置瓶颈段）
   - 数据冲突（分类、定向、暂停）
   - 控制冲突（分支改进、分支延迟）
4. 向量计算机
   - 向量数据表示
   - 向量流水线

第三章作业

王志英教材

第111页： T2、T3、T4、T5。

和第四章作业一起交。

课程报告

以下报告内容均需要引用本课程的相关内容：

- 选择你认为计算机发展历史上最重要的十台机器，并说明理由。
- 由于AI的广泛应用，CPU+GPU的架构越来越普遍，比较各厂家架构设计上的特点。
- 近五年来，在CPU、存储、总线、IO 等方面有什么重要的新进展，分析它的影响和未来的发展。

选择以上三个主题中的一个来完成报告

第 0 章 前言

第 1 章 计算机体系结构基本概念

第 2 章 指令系统

第 3 章 流水线技术

第 4 章 指令级并行

第 5 章 存储层次

第 6 章 输入输出系统

第四章 指令级并行

4.1 指令级并行的概念

4.2 指令的动态调度

4.3 控制相关的动态解决技术

4.4 多指令流出技术

4.1 指令级并行的概念

4.1.1 循环展开调度的基本方法

4.1.2 相关性

4.1 指令级并行的概念

- 当指令之间不存在相关时，它们在流水线中是可以重叠起来并行执行的。这种指令序列中存在的潜在并行性称为指令级并行
  - **Instruction-Level Parallelism**
  - 简记为**ILP**
- 如何知道指令之间可以并行？硬、软件如何支持指令级并行？如何研究这些问题？
  - 硬件技术或者软件技术都可以提高指令级并行性
  - 必须要硬件技术和软件技术互相配合，才能够最大限度地挖掘出程序中存在的指令级并行

性能评价：CPI计算

- 流水线处理器的实际**CPI**（平均每条指令使用的周期数）等于理想流水线的**CPI**加上各类停顿引起的周期数的总和

$$
\begin{aligned}
CPI_{流水线} &= CPI_{理想} \\
&+ 停顿_{结构冲突} \\
&+ 停顿_{先写后读} \\
&+ 停顿_{先读后写} \\
&+ 停顿_{写后写} \\
&+ 停顿_{控制冲突}
\end{aligned}
$$

- 减少其中的任何一种停顿，都可以有效地减少**CPI**，从而提高流水线的性能

本章研究的技术及克服的停顿

技术	主要克服的停顿	相关章节
基本流水线调度	数据先写后读相关停顿	**4.1**
循环展开	控制相关停顿	**4.1**
寄存器换名	数据写后写和先读后写相关停顿	**4.1**
指令动态调度（记分牌和Tomasulo算法）	各种数据相关停顿	**4.2**
动态分支预测	控制相关停顿	**4.3**
前瞻（Speculation）	所有数据/控制相关停顿	**4.3**
多指令流出（超标量和超长指令字）	提高理想CPI	**4.4**

软件和硬件的支持

- 上述技术中有些技术主要是硬件支持
  - 循环展开
  - 寄存器换名的动态调度（基本的**Tomasulo’s**）
  - 动态分支指令预测
  - 前瞻技术
  - 每个周期多发射
- 所有的技术都必须和软件，特别是编译器合作完成

几个基本概念

- 基本（程序）块：一段除了入口和出口以外不包含其它分支的线性代码段
  - 程序平均每6~7条指令就会有一个分支
  - 必须在多个基本块之间开发指令级的并行性
- 循环级并行：开发循环体的不同迭代之间存在的并行性
- 开发循环级并行的基本技术方法
  - 指令调度（scheduling）
  - 循环展开（loop unrolling）
  - 换名（renaming）

4.1 指令级并行的概念

4.1.1 循环展开调度的基本方法

4.1.2 相关性

4.1.1 循环展开调度的基本方法

- 循环展开是展开循环体若干次，将循环级并行转化为指令级并行的技术
- 这个过程既可以通过编译器静态完成，也可以通过硬件动态进行
  - 开发循环级并行性的另外一个重要技术是向量处理技术
  - 具有向量处理指令的典型机器是向量计算机，有关向量处理和向量计算机的内容本章不作讨论

- 本章中的分支指令就是指条件转移指令

本章通用浮点流水线延迟表

- 编译器在完成这种指令调度时，受限于以下两个特性：
  - 一是程序固有的指令级并行性
  - 二是流水线功能部件的执行延迟

- 本章中使用的浮点流水线的延迟如下表

产生结果指令	使用结果指令	延迟时钟数
浮点计算	另外的浮点计算	3
浮点计算	浮点数据存操作（SD）	2
浮点数据取操作（LD）	浮点计算	1
浮点数据取操作（LD）	浮点数据存操作（SD）	0

整数流水线其他特性说明

- 整数流水线采用改进的MIPS整数流水线
  - 载入延迟为1个节拍
  - 由于数据的取操作的结果可以毫无停顿的通过相关通路机制传送到数据存部件，所以延迟为0
  - 有定向通道或旁路机制
- 分支指令，由整数流水线执行
  - 分支条件检测调整到ID段
  - 如果分支指令使用上一条指令的结果作为分支条件，将要延迟1节拍
  - 分支指令有1个节拍的延迟槽
- 浮点运算一般为64位

循环展开实例

- 对于下面的源代码，在不进行指令调度和进行指令调度两种情况下，分析代码一次循环的执行时间。

```
for (i=1; i<=1000; i++)
    x[i] = x[i] + s;
```

MIPS汇编语言程序

- 程序转换成MIPS汇编语言程序

```
Loop:   LD      F0,0(R1)    ;F0为向量元素
        ADDD    F4,F0,F2    ;加常数F2
        SD      0(R1),F4    ;保存结果
        SUBI    R1,R1,#8    ;修改指针
        BNEZ    R1,Loop     ;循环控制
```

循环无调度执行

时钟	无调度
1	LD F0,0(R1)
2	? ADDD F4,F0,F2

产生结果指令	使用结果指令	延迟时钟数
浮点计算	另外的浮点计算	3
浮点计算	浮点数据存操作（SD）	2
浮点数据取操作（LD）	浮点计算	1
浮点数据取操作（LD）	浮点数据存操作（SD）	0

循环无调度执行

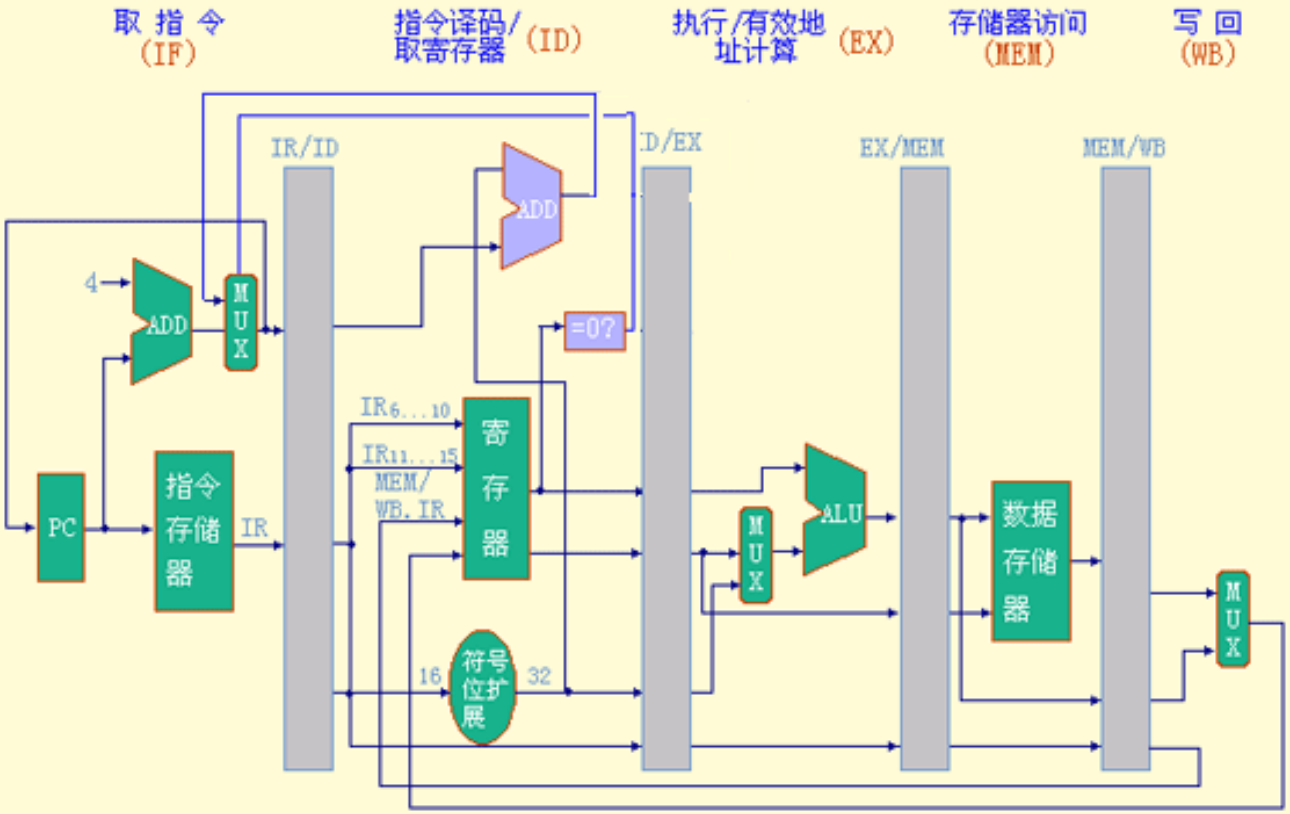
时钟	无调度
1	LD F0,0(R1)
2	*stall*
3	ADDD F4,F0,F2
4	? SD 0(R1),F4

产生结果指令	使用结果指令	延迟时钟数
浮点计算	另外的浮点计算	3
浮点计算	浮点数据存操作（SD）	2
浮点数据取操作（LD）	浮点计算	1
浮点数据取操作（LD）	浮点数据存操作（SD）	0

循环无调度执行

时钟	无调度
1	LD F0,0(R1)
2	*stall*
3	ADDD F4,F0,F2
4	*stall*
5	*stall*
6	SD 0(R1),F4
7	? SUBI R1,R1,#8

修改 DLX 以减少分支开销



循环无调度执行

时钟	无调度
1	LD F0,0(R1)
2	*stall*
3	ADDD F4,F0,F2
4	*stall*
5	*stall*
6	SD 0(R1),F4
7	SUBI R1,R1,#8
8	? BNEZ R1,LOOP

如果分支指令使用上一条指令的结果作为分支条件，将要延迟1节拍

循环无调度执行

时钟	无调度
1	LD F0,0(R1)
2	*stall*
3	ADDD F4,F0,F2
4	*stall*
5	*stall*
6	SD 0(R1),F4
7	SUBI R1,R1,#8
8	*stall*
9	BNEZ R1,LOOP

循环无调度执行

时钟	无调度
1	LD F0,0(R1)
2	*stall*
3	ADDD F4,F0,F2
4	*stall*
5	*stall*
6	SD 0(R1),F4
7	SUBI R1,R1,#8
8	*stall*
9	BNEZ R1,LOOP
10	?

分支指令有1个节拍的延迟槽

循环无调度执行

时钟	无调度
1	LD F0,0(R1)
2	*stall*
3	ADDD F4,F0,F2
4	*stall*
5	*stall*
6	SD 0(R1),F4
7	SUBI R1,R1,#8
8	*stall*
9	BNEZ R1,LOOP
10	*stall*

循环无调度执行结果分析

- 每遍循环需要10个时钟节拍
- 只有3个时钟节拍(LD, ADDD, SD)
  - 有效比率为30%
  - 空转5个时钟节拍 (50%)
  - 循环控制2个时钟节拍 (20%)

- 调度代码，减少空转

调度代码

时钟	未调度	调度后
1	LD F0,0(R1)	LD F0,0(R1)
2	*Stall*	
3	ADDD F4,F0,F2	
4	*Stall*	
5	*Stall*	
6	SD 0(R1),F4	
7	SUBI R1,R1,#8	
8	*Stall*	
9	BNEZ R1,Loop	
10	*Stall*	

调度代码

时钟	未调度	调度后
1	LD F0,0(R1)	LD F0,0(R1)
2	*Stall*	?
3	ADDD F4,F0,F2	
4	*Stall*	
5	*Stall*	
6	SD 0(R1),F4	
7	SUBI R1,R1,#8	
8	*Stall*	
9	BNEZ R1,Loop	
10	*Stall*	

调度代码

时钟	未调度	调度后
1	LD F0,0(R1)	LD F0,0(R1)
2	*Stall*	SUBI R1,R1,#8
3	ADDD F4,F0,F2	
4	*Stall*	
5	*Stall*	
6	SD 0(R1),F4	
7	SUBI R1,R1,#8	
8	*Stall*	
9	BNEZ R1,Loop	
10	*Stall*	

调度代码

时钟	未调度	调度后
1	LD F0,0(R1)	LD F0,0(R1)
2	*Stall*	SUBI R1,R1,#8
3	ADDD F4,F0,F2	ADDD F4,F0,F2
4	*Stall*	??
5	*Stall*	??
6	SD 0(R1),F4	SD 0(R1),F4
7	SUBI R1,R1,#8	
8	*Stall*	
9	BNEZ R1,Loop	
10	*Stall*	

调度代码

时钟	未调度	调度后
1	LD F0,0(R1)	LD F0,0(R1)
2	*Stall*	SUBI R1,R1,#8
3	ADDD F4,F0,F2	ADDD F4,F0,F2
4	*Stall*	*Stall*
5	*Stall*	BNEZ R1,Loop
6	SD 0(R1),F4	SD 8(R1),F4
7	SUBI R1,R1,#8	
8	*Stall*	
9	BNEZ R1,Loop	
10	*Stall*	

如何进行调度

- 通过一个好的编译器来调度可以
  - 调换**SUBI**和**SD**的位置
  - 将**SD**指令移到**BNEZ**的延迟槽内
  - 改变**SD**存取指令访问内存地址的偏移量

调度代码结果分析

- 每遍循环6个时钟节拍
- 和未调度代码比较，加速比 10/6=1.7
- 3个有效时钟节拍 (LD, ADDD, SD)
  - 节拍有效比率50%
  - 1拍空转 (占17%)
  - 2拍循环控制 (占33%)

- 如何进一步减少空转和循环控制占用的比率?

调度代码结果分析

- 每遍循环6个时钟节拍
- 和未调度代码比较，加速比 10/6=1.7
- 3个有效时钟节拍 (LD, ADDD, SD)
  - 节拍有效比率50%
  - 1拍空转 (占17%)
  - 2拍循环控制 (占33%)

- 如何进一步减少空转和循环控制占用的比率?

循环展开

- 如果将循环展开3次得到4个循环体(假设数组是4的倍数的元素)

```
Loop:   LD      F0,0(R1)
        ADDD    F4,F0,F2
        SD      0(R1),F4
        SUBI    R1,R1,#8
        LD      F0, 0(R1)
        ADDD    F4,F0,F2
        SD      0(R1), F4
        SUBI    R1,R1,#8
        LD      F0, 0(R1)
        ADDD    F4,F0,F2
        SD      0(R1), F4
        SUBI    R1,R1,#8
        LD      F0,0(R1)
        ADDD    F4,F0,F2
        SD      0(R1), F4
        SUBI    R1,R1,#8
        BNEZ    R1,Loop
```

- 编译器直接调整存储器访问的偏移量
- 大量关于F0,F4的名相关，指令难以调度

循环展开

- 如果将循环展开3次得到4个循环体(假设数组是4的倍数的元素)

```
Loop:   LD      F0,0(R1)
        ADDD    F4,F0,F2
        SD      0(R1),F4
        LD      F6,-8(R1)
        ADDD    F8,F6,F2
        SD      -8(R1), F8
        LD      F10,-16(R1)
        ADDD    F12,F10,F2
        SD      -16(R1), F12
        LD      F14,-24(R1)
        ADDD    F16,F14,F2
        SD      -24(R1), F16
        SUBI    R1,R1,#-32
        BNEZ    R1,Loop
```

循环无调度执行

时钟	无调度
1	LD F0,0(R1)
2	*stall*
3	ADDD F4,F0,F2
4	*stall*
5	*stall*
6	SD 0(R1),F4
7	SUBI R1,R1,#8
8	*stall*
9	BNEZ R1,LOOP
10	*stall*

执行时间分析

Loop:	LD	F0,0(R1)	1
	ADDD	F4,F0,F2	2,3
	SD	0(R1),F4	4,5,6
	LD	F6,-8(R1)	7
	ADDD	F8,F6,F2	8,9
	SD	-8(R1), F8	10,11,12
	LD	F10,-16(R1)	13
	ADDD	F12,F10,F2	14,15
	SD	-16(R1), F12	16,17,18
	LD	F14,-24(R1)	19
	ADDD	F16,F14,F2	20,21
	SD	-24(R1), F16	22,23,24
	SUBI	R1,R1,#-32	25
	BNEZ	R1,Loop	26,27
	stall		28

结果分析

- 循环使用28个时钟节拍
  - 14个空转节拍
    - 每个LD有1个空转个节拍 – 共4拍
    - 每个ADDD有2个空转节拍 - 共8拍
    - SUBI有1个空转节拍 - 共1拍
    - BRANCH有1个空转节拍 - 共1拍
  - 有14个指令流出节拍
- 平均每遍循环7个时钟节拍
- 共计使用9个寄存器

循环展开+指令调度

```
1.  Loop:   LD      F0,0(R1)
2.          LD      F6,-8(R1)
3.          LD      F10,-16(R1)
4.          LD      F14,-24(R1)
5.          ADDD    F4,F0,F2
6.          ADDD    F8,F6,F2
7.          ADDD    F12,F10,F2
8.          ADDD    F16,F14,F2
9.          SD      0(R1),F4
10.         SD      -8(R1),F8
11.         SUBI    R1,R1,#-32
12.         SD      16(R1),F12
13.         BNEZ    R1,R2,Loop
14.         SD      8(R1),F16
```

“循环展开+指令调度”结果分析

- 每遍循环时间下降为**14**个时钟节拍
  - 每个元素平均使用**3.5**个时钟节拍
- 比较
  - 没有循环展开，有指令调度
    - 每个元素**6**拍
  - 有循环展开，没有指令调度
    - 每个元素**7**拍

循环展开和指令调度的总结

- 保证正确性（循环控制和操作数偏移量的修改）
- 注意有效性（找到不同循环体之间的无关性）
- 使用大量不同的寄存器（换名）
- 减少循环控制中的测试指令和分支指令
- 注意分析**Load/Store**指令的内存地址
- 注意新的相关性

4.1 指令级并行的概念

4.1.1 循环展开调度的基本方法

4.1.2 相关性

4.1.2 相关性

- **相关与流水线冲突 (Dependence and Hazard)**
  - **相关：**两条指令之间存在某种依赖关系。
    如果两条指令相关，则它们就有可能不能在流水线中重叠执行或者只能部分重叠执行。
  - 相关有3种类型
    - 数据相关（也称真数据相关、数据流相关）
    - 名相关（包括反相关、输出相关）
    - 控制相关

4.1.2 相关性

1. 数据相关、真数据相关、数据流相关

（Data dependence、Flow Dependence）

– 对于两条指令i（在前）和j（在后），如果下述条件之一成立，则称指令j与指令i数据相关。

(1)指令j使用指令i产生的结果；

(2)指令j与指令k数据相关，而指令k又与指令i数据相关。

– 数据相关具有传递性。

数据相关是两条指令存在先写后读的相关链

引起流水线的RAW冲突

例如：下面这一段代码存在数据相关。(数组增值)

```
Loop:  LD      F0, 0 (R1)      // F0为数组元素
       ADDD    F4, F0, F2      // 加上F2中的值
       SD      0 (R1), F4      // 保存结果
       SUBI    R1, R1, #8      // 数组指针递减8个字节
       BNEZ    R1, Loop        // 如果R1≠0，则分支
```

- 如果两条指令之间有真数据相关，那么它们就不能同时执行或是完全重叠执行
- 同时执行这些指令会造成正在流水的处理机检测到这种冲突并插入暂停，从而减少甚至取消指令之间的重叠
- 指令序列中存在的真数据相关反映出产生该指令序列的程序源代码的相关关系

相关性是程序的一个特性，
是否一个相关会导致实际的冲突，是否该冲突会造成暂停，
这是流水线结构的基本特性。

```
Loop:  LD      F0, 0 (R1)      // F0为数组元素
       ADDD    F4, F0, F2      // 加上F2中的值
       SD      0 (R1), F4      // 保存结果
       SUBI    R1, R1, #8      // 数组指针递减8个字节
       BNEZ    R1, Loop        // 如果R1≠0，则分支
```

- 如果分支检测移到了ID流水段，相关会造成1次暂停
- 如果分支检测仍在EX流水段，这个相关就不会引起暂停

- 数据相关性会限制可以开发的指令级并行性
- 解决方法
  - 硬件：采用互锁机制，一旦检测存在数据相关，则停止本指令的执行，并插入空转周期
  - 软件：使用编译器，在相关出插入空操作（空转周期），保证当前指令不会错误的使用一个尚未产生的数据。
- 数据相关的检测
  - 当数据的流动是经过寄存器时，相关的检测比较直观和容易
  - 当数据的流动是经过存储器时，检测比较复杂。
    - 相同形式的地址其有效地址未必相同；
    - 形式不同的地址其有效地址却可能相同。

4.1.2 相关性

2. 名相关（**Name dependence**）
   - 名：指令所访问的寄存器或存储器单元的名称。
   - 如果两条指令使用相同的名，但是它们之间并没有数据流动，则称这两条指令存在名相关。

– 指令j与指令i之间的名相关有两种：

（1）反相关（Anti dependence）：

如果指令j写的名与指令i读的名相同（引发流水线的先读后写WAR冲突），则称指令i和j发生了反相关。

指令j写的名＝指令i读的名

必须保证指令的原来顺序，以确保i能读到正确的值。

（2）输出相关（Output dependence）：

如果指令j和指令i写相同的名（引发流水线的写后写WAW冲突），则称指令i和j发生了输出相关。

指令j写的名＝指令i写的名

操作顺序必须得到保证，以使最后写入的值得以保存。

2. 名相关

- 两条指令之间并没有数据的传送。
- 如果一条指令中的名改变了，并不影响另外一条指令的执行。
- 换名技术
  - 通过改变指令中操作数的名来消除名相关。
  - 对于寄存器操作数进行换名称为寄存器换名（重命名，Renaming）。
  - 既可以用编译器静态实现，也可以用硬件动态完成。

循环展开中的换名

- 如果将循环展开3次得到4个循环体(假设数组是4的倍数的元素)

```
Loop:   LD    F0,0(R1)
        ADDD  F4,F0,F2
        SD    0(R1),F4
        LD    F6,-8(R1)
        ADDD  F8,F6,F2
        SD    -8(R1), F8
        LD    F10,-16(R1)
        ADDD  F12,F10,F2
        SD    -16(R1), F12
        LD    F14,-24(R1)
        ADDD  F16,F14,F2
        SD    -24(R1), F16
        SUBI   R1,R1,#-32
        BNEZ  R1,Loop
```

– 再考虑以下代码：

```
DIVD    F0，F2，F4
MULD    F6，F0，F8
SD      0（R1），F6
SUBD    F8，F10，F14
ADDD    F6，  F10，F8
```

输出相关（F6）导致WAW冲突

反相关（F8）导致WAR冲突

第二、三条指令中的F6是真数据相关
第四、五条指令中的F8是真数据相关

– 消除名相关

- 引入两个临时寄存器S和T
- 把这段代码改写为：

```
DIVD    F0，F2，F4
MULD    S，F0，F8
SD      0（R1），S
SUBD    T，F10，F14
ADDD    F6，F10，T
```

两个F6都换名为S

两个F8都换名为T

Dependence Types

Flow dependence

$r_3 \leftarrow r_1 \text{ op } r_2$

$r_5 \leftarrow r_3 \text{ op } r_4$

Read-after-Write (RAW)

Anti dependence

$r_3 \leftarrow r_1 \text{ op } r_2$

$r_1 \leftarrow r_4 \text{ op } r_5$

Write-after-Read (WAR)

Output-dependence

$r_3 \leftarrow r_1 \text{ op } r_2$

$r_5 \leftarrow r_3 \text{ op } r_4$

$r_3 \leftarrow r_6 \text{ op } r_7$

Write-after-Write (WAW)

4.1.2 相关性

3. 控制相关
   - 由分支指令引起的相关。
     - 为了保证程序应有的执行顺序，必须严格按控制关确定的顺序执行。
   - 典型的程序结构是“if-then”结构。
   - 例：

```
if p1 {
        S1;
    };
S;

if p2 {
        S2;
    };
```

– 控制相关有以下两个原则：

（1）与控制相关的指令不能移到分支指令之前，即控制有关的指令不能调度到分支指令的控制范围之外。

- 对于上述的例子，then 部分中的指令不能移到if语句之前

（2）与控制无关的指令不能移到分支指令之后，即控制无关的指令不能调度到分支指令的控制范围以内。

- 对于上述的例子，不能把S移到if语句的then 部分中。

```
if p1 {
        S1;
    };

S;

if p2 {
        S2;
    };
```